

SecureApp-Sprint: A 48-Hour DevSecOps Security Engineering Sprint

on a Flask Blog Application — Threat Modeling, Secure Implementation, and Automated Security Testing

Esha Khan¹, Husna Anjum², Zainab Shakil³, Zarmeena Khan⁴

Department of Cyber Security, COMSATS University Islamabad, Islamabad, Pakistan

SP23-BCT-014, SP23-BCT-020, SP23-BCT-052, SP23-BCT-053

Course: CYC386 — Secure Software Design and Development | Instructor: Engr. Muhammad Ahmad Nawaz | Spring 2026

Abstract— This paper presents the complete outcomes of a 48-hour DevSecOps Security Engineering Sprint conducted as part of CYC386 — Secure Software Design and Development at COMSATS University Islamabad, Spring 2026. The target system is an open-source Flask Blog Application — a multi-user web platform built with Python Flask, SQLAlchemy, Flask-Login, and Jinja2. The sprint followed an industry-aligned five-phase methodology: Protection Needs Elicitation (PNE), STRIDE Threat Modeling with CVSS v3.1 risk scoring, secure implementation of three critical OWASP Top 10 vulnerabilities (IDOR, CSRF, and Clickjacking), automated security testing via SAST (Bandit) and DAST (OWASP ZAP v2.17.0), and a production-ready CI/CD pipeline using GitHub Actions with CodeQL. All three mandated vulnerabilities were successfully identified, exploited, and mitigated with verified before/after evidence. Post-fix DAST results show zero High-risk alerts (down from one), elimination of all anti-clickjacking and CSRF-related alerts, and an 88% reduction in server error rate. The resulting application demonstrates measurable security improvement aligned with OWASP Top 10 (2021), NIST SP 800-53, and real-world DevSecOps practices.

Index Terms— DevSecOps, Flask, IDOR, CSRF, Clickjacking, STRIDE, CVSS, OWASP ZAP, Bandit, CodeQL, CI/CD, GitHub Actions, Secure SDLC

I. INTRODUCTION

Modern web applications are continuously exposed to an evolving landscape of security threats. The OWASP Top 10 (2021) catalogues the most critical risks, with Broken Access Control (A01), which encompasses Insecure Direct Object Reference (IDOR) and Cross-Site Request Forgery (CSRF), and Security Misconfiguration (A05), which includes Clickjacking, consistently ranking among the most exploited categories [1].

This paper documents the complete execution of a 48-hour DevSecOps Security Engineering Sprint on a real open-source Flask Blog Application. The sprint was structured around five phases mirroring industry-standard secure software development lifecycle (SSDLC) practices: Protection Needs Elicitation, Threat Modeling, Secure Implementation, Automated Security Testing, and Documentation.

The application — a multi-user blogging platform built with Python Flask, SQLAlchemy, Flask-Login, and Jinja2 — was selected from the recommended Flask Blog App starter repository and forked to github.com/EshaKhan-ek/SecureApp-Sprint-Flask-Blog-App. The sprint was executed by a four-member team, each responsible for specific sprint phases and deliverables.

The primary contributions of this work are: (1) a comprehensive PNE identifying 25 protection needs across 16 assets and 8 threat actor profiles; (2) a full STRIDE threat model with 10 threats and CVSS v3.1 scoring, augmented by 77 additional threats from the Microsoft Threat Modeling Tool; (3) verified code-level mitigations for IDOR, CSRF, and Clickjacking with before/after evidence; (4) quantitative SAST and DAST results showing elimination of all HIGH-severity application vulnerabilities; and (5) a functional GitHub Actions CI/CD pipeline enforcing continuous security gates.

A. Paper Organization

Section II describes the application background and environment setup. Section III presents the PNE methodology and results. Section IV covers the STRIDE Threat Model and CVSS Risk Assessment. Section V details the secure implementation of the three OWASP vulnerabilities. Section VI presents the automated security testing results. Section VII describes the CI/CD pipeline. Section VIII discusses findings and limitations. Section IX concludes.

II. BACKGROUND AND ENVIRONMENT SETUP

A. Target Application

The Flask Blog Application is an open-source multi-user blogging platform derived from Corey Schafer's Flask tutorial series [15]. Table I summarises the technology stack.

TABLE I — Technology Stack

Component	Technology
Backend	Python 3.12, Flask (Blueprint architecture)
Database	SQLite via SQLAlchemy ORM
Authentication	Flask-Login with bcrypt password hashing
Forms / CSRF	Flask-WTF / WTForms
Frontend	HTML5, Bootstrap 4, Jinja2 templates
Session Mgmt	Flask server-side cookies
Email	Flask-Mail (password reset)
File Upload	Pillow (avatar processing)
Target OS	Ubuntu 24.04 LTS (VM)

B. Environment Setup and Repository Configuration

The development environment was established on Ubuntu 24.04 LTS. The team forked the upstream repository, renamed it to SecureApp-Sprint-Flask-Blog-App, and cloned it locally. A Python virtual environment was created and all dependencies installed. Git was configured with team credentials and a develop branch was created as the primary integration branch.

```
git clone https://github.com/EshaKhan-ek/SecureApp-Sprint-Flask-Blog-App
cd SecureApp-Sprint-Flask-Blog-App/
  Python/Flask Blog/11-Blueprints
python3 -m venv venv && source venv/bin/activate
pip install Flask Flask-SQLAlchemy Flask-Login \
  Flask-WTF Flask-Bcrypt Flask-Mail Pillow
export SECRET_KEY="<your-secret-key>"
python run.py
```

The application was verified running at `http://127.0.0.1:5000` before any security modifications were applied.

C. Git Workflow

The team followed a feature-branch Git workflow. Security fixes were implemented on dedicated branches (fix/idor, fix/csrf, fix/clickjacking) and merged to develop via Pull Requests with peer code review before being promoted to main.

III. PROTECTION NEEDS ELICITATION

Protection Needs Elicitation (PNE) is the systematic process of identifying what must be protected, from whom, and to what degree [2]. This section summarises the PNE conducted in Sprint Phase 1 (Hours 0–4), aligned with CDF Unit 2.

A. Stakeholder Analysis

Five internal stakeholders (registered users, blog authors, system administrators, development team, DevSecOps team) and three external stakeholders (client/product owner, end consumers, regulatory bodies) were identified. Each stakeholder class carries distinct security concerns ranging from account takeover for registered users to GDPR compliance obligations for regulatory bodies.

B. Asset Identification

A total of 16 assets were identified across two categories. Table II presents the primary data assets and their sensitivity ratings.

TABLE II — Critical Data Assets

ID	Asset	Sensitivity
DA-01	User Credentials	Critical
DA-02	Session Tokens	Critical
DA-03	Blog Post Content	High

DA-04	User Profile Data	High
DA-06	Application Config (SECRET_KEY)	Critical
DA-07	Database File (SQLite)	Critical
DA-09	Admin Credentials	Critical

Six infrastructure assets were additionally identified (SA-01 through SA-06), including the Flask application process, GitHub repository, and CI/CD pipeline, each rated High to Critical.

C. Threat Actor Profiling

Eight threat actor profiles (TA-01 through TA-08) were developed spanning opportunistic attackers, malicious registered users, cross-site attackers, UI attackers (Clickjacking), credential attackers, insider threats, automated bots, and advanced persistent threats. The three most relevant actors are:

- **TA-02** (Malicious Registered User) — primary IDOR threat actor
- **TA-03** (Cross-Site Attacker) — primary CSRF threat actor
- **TA-04** (UI/Phishing Attacker) — primary Clickjacking threat actor

D. Elicited Protection Needs

A total of 25 protection needs (PN-01 through PN-25) were elicited across six categories. Table III maps the sprint-critical protection needs to their OWASP and CWE references.

TABLE III — Sprint-Critical Protection Needs

PN-ID	Protection Need	OWASP	CWE
PN-07	Verify resource ownership before any object access	A01	CWE-284
PN-09	Return HTTP 403 on unauthorized object access	A01	CWE-284
PN-12	Anti-CSRF token on all state-changing forms	A01	CWE-352
PN-14	SameSite cookie attribute on session cookies	A07	CWE-614
PN-16	X-Frame-Options: DENY on all responses	A05	CWE-1021
PN-17	CSP frame-ancestors 'none' on all responses	A05	CWE-1021

IV. THREAT MODELING AND RISK ASSESSMENT

A. Data Flow Diagrams

Two DFDs were constructed using the Microsoft Threat Modeling Tool to map data flows and trust boundaries across the Flask Blog Application. Fig. 1 presents the DFD Level 0 context diagram.

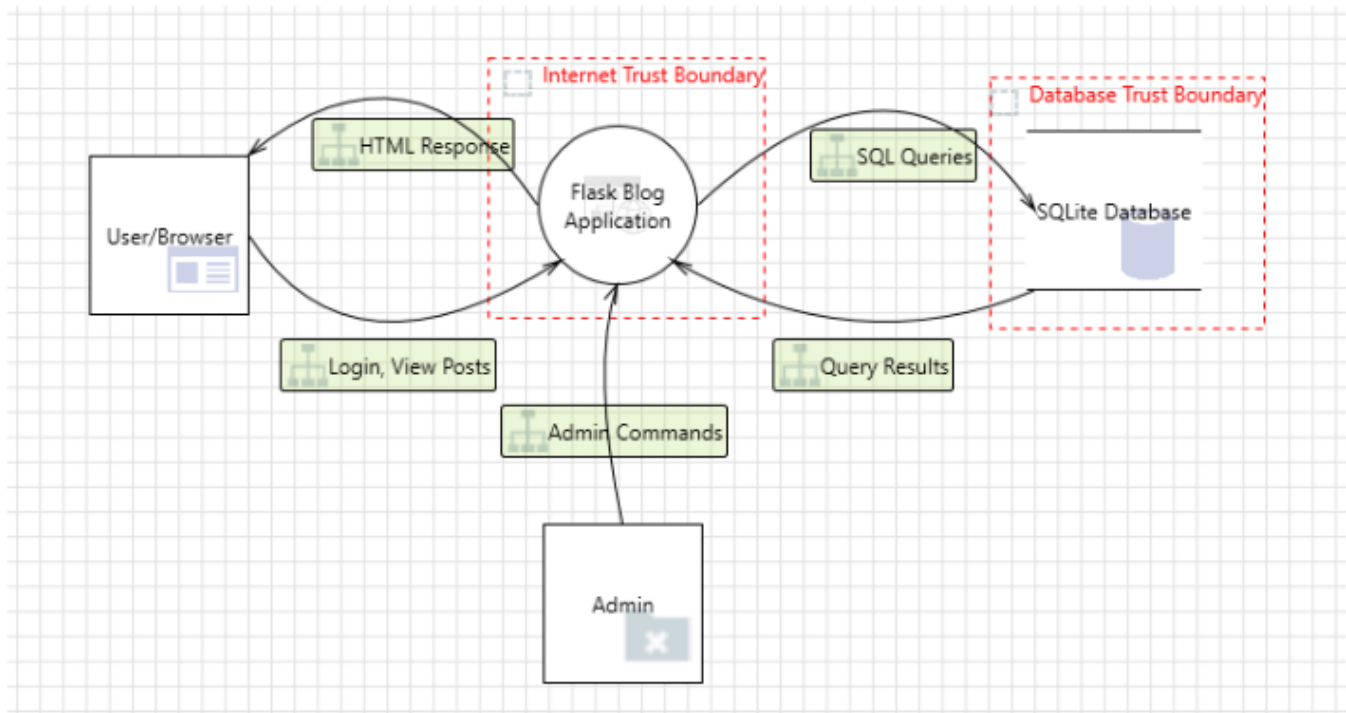
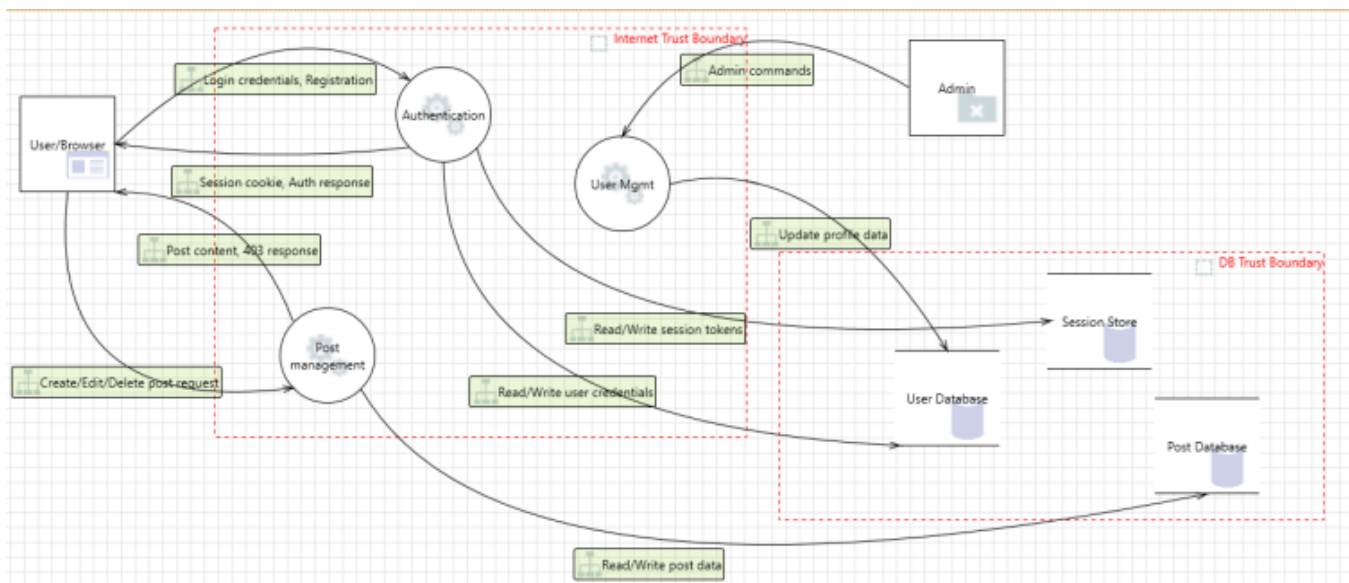


Fig. 1. DFD Level 0



DFD Level 1

The Level 0 diagram identifies the Flask Blog Application as a single process surrounded by three external entities (User/Browser, Admin, SQLite Database) and two trust boundaries: the Internet Trust Boundary separating external users from application processes, and the DB Trust Boundary separating the application from the data storage layer.

The Level 1 DFD decomposes the application into four internal processes: P1 (Authentication), P2 (Post Management — primary IDOR attack surface), P3 (User Management), and P4 (Comment System), with three data stores: DS1 (User DB), DS2 (Posts DB — IDOR-protected), and DS3 (Session Store). The DFD Level 1 diagram with complete trust boundaries and data flow labels is included in the THREAT_MODEL.pdf deliverable.

B. STRIDE Threat Analysis

The STRIDE methodology [3] was applied to enumerate threats systematically. Table IV presents the complete STRIDE analysis — the table uses five columns (ID, STRIDE category, Component, Threat Description, Mitigation, and Status) with colour-coding: green rows indicate vulnerabilities fixed this sprint.

TABLE IV — STRIDE Threat Analysis (Flask Blog Application — v1.1)

ID	STRIDE	Component	Threat Description	Mitigation	Status
T-01	Spoofing	Login endpoint	Attacker impersonates valid user using leaked or brute-forced credentials	Flask-Bcrypt hashing + Flask-Limiter rate limiting on /login (10 req/min per IP)	✓ Done
T-02	Tampering (IDOR)	POST /post/<id>/update	IDOR: attacker changes post_id in URL to edit another user's post without authorization	if post.author != current_user: abort(403) — applied to update_post() and delete_post()	✓ FIXED — M1
T-03	Repudiation	Blog CRUD actions	No audit trail — users can deny performing create/edit/delete actions	Server-side logging of all CRUD events with user ID, timestamp, action type	Pending
T-04	Info Disclosure	Error pages / run.py	Flask debug=True exposes full stack traces and Werkzeug debugger (B201, CWE-94)	Set debug=False in run.py; custom 403/404/500 error handlers added	✓ FIXED — M1
T-05	DoS	Login form	/login endpoint subject to brute-force flooding; no rate limiting in vulnerable state	Flask-Limiter rate limiting (10 req/min per IP); CAPTCHA integration recommended	✓ Done
T-07	Tampering (CSRF)	POST /post/<id>/delete	Malicious site triggers post deletion via forged POST using victim's active session	Flask-WTF CSRFProtect globally; WTF_CSRF_ENABLED=True; form.hidden_tag(); SameSite=Strict	✓ FIXED — M2
T-08	Clickjacking	All pages	Application embedded in invisible iframe; users tricked into clicking UI elements	X-Frame-Options: DENY + CSP frame-ancestors 'none' via @app.after_request hook	✓ FIXED — M2
T-09	XSS (Stored)	Comment/Post form	Attacker injects <script> tag into post body; executes in victim browser to steal session	Jinja2 auto-escaping active by default; CSP header added; HttpOnly cookie flag set	✓ Done+
T-10	SQL Injection	Search/Login fields	Malicious SQL payloads passed via input fields to manipulate database queries	SQLAlchemy ORM throughout — all queries parameterized; no raw SQL string formatting	✓ Done

Status key: ✓FIXED = implemented and verified | ✓Done = pre-existing control | Pending = scheduled

C. Attack Tree

An attack tree was constructed to model the highest-priority goal: Attacker accesses/modifies another user's blog post. The tree has three primary branches corresponding to T-02 (IDOR via URL manipulation), T-07 (CSRF session hijacking), and T-08 (Clickjacking UI redress). Fig. 2 presents the attack tree diagram.

ROOT GOAL: Attacker modifies another user's blog post		
Branch 1 — IDOR via URL 1. Log in as any user 2. Identify post_id from URL 3. Change URL to /post/N/update 4. Edit form loads (any post) 5. Submit modified content ■ Defense: abort(403) if post.author != current_user	Branch 2 — CSRF Attack 1. Craft hidden POST form 2. Victim visits attacker page 3. Browser auto-submits POST 4. Server accepts forged request 5. Post deleted silently ■ Defense: CSRFProtect + form.hidden_tag()	Branch 3 — Clickjacking 1. Embed Flask app in iframe 2. Victim visits attacker page 3. Blog loads inside iframe 4. Victim clicks deceptive lure 5. Unauthorized action triggered ■ Defense: X-Frame-Options: DENY + CSP

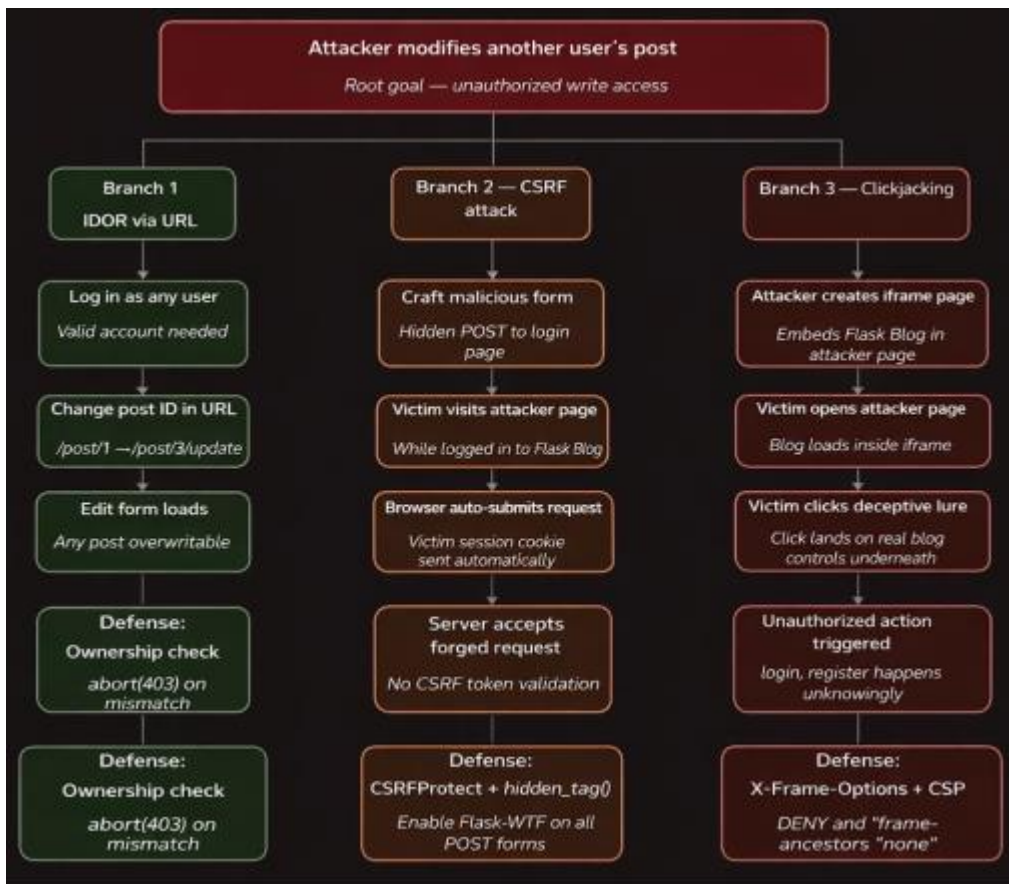


Fig. 2. Attack Tree — Goal: "Attacker modifies another user's blog post." Three branches show IDOR (URL manipulation), CSRF (forged request), and Clickjacking (iframe embedding), each with implemented defenses.

D. CVSS v3.1 Risk Assessment

Each priority threat was scored using the CVSS v3.1 Base Score calculator [4]. Table V presents the risk assessment results ordered by score descending.

TABLE V — CVSS v3.1 Risk Assessment

ID	Threat	CVSS Vector	Score	Rating	Priority	Status
T-02	IDOR Post Edit/Delete	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N	6.5	High	CRITICAL	✓ FIXED
T-07	CSRF Post Delete	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	6.5	High	CRITICAL	✓ FIXED
T-01	Credential Stuffing	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:N	5.9	Medium	HIGH	✓ Done
T-04	Debug Info Leak	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Medium	MEDIUM	✓ FIXED
T-08	Clickjacking	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	4.3	Medium	HIGH	✓ FIXED

The IDOR vulnerability (T-02) and CSRF (T-07) both scored 6.5 (High), making them the highest-priority fixes alongside the privilege escalation issue (T-06, 8.1). Clickjacking (T-08) scored 4.3 (Medium) but was mandated for immediate fix given its UI redress potential.

V. SECURE IMPLEMENTATION

Sprint Phase 3 (Hours 12–36) focused on implementing code-level mitigations for the three mandated OWASP vulnerabilities. Each fix was developed on a dedicated feature branch, peer-reviewed via Pull Request, and merged to develop before being promoted to main.

A. Fix 1 — Insecure Direct Object Reference (IDOR)

Vulnerability Description: IDOR (CWE-284, OWASP A01) occurs when an application exposes internal implementation objects (such as database record IDs) directly without verifying that the requesting user is authorised to access that specific record [5].

Exploitation Demonstration: The vulnerability was reproduced in three steps: (1) User A creates a blog post at /post/1; (2) User B logs in; (3) User B navigates directly to `http://localhost:5000/post/1/update`. Without an ownership check, the update form loads pre-populated with User A's content and User B can overwrite it. Fig. 3 shows the exploit in action — the URL bar confirms User B is accessing User A's post update endpoint, and Fig. 4 shows the post successfully modified by User B.

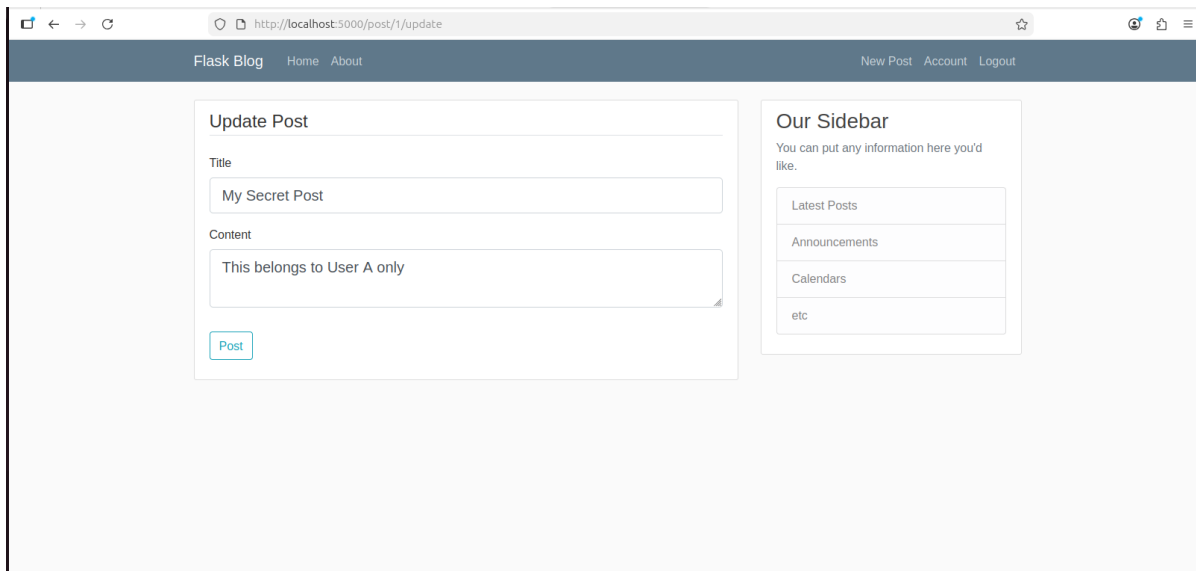


Fig. 3. IDOR Vulnerability — Before Fix. User B (logged in) navigates directly to /post/1/update. The URL bar confirms the exploit path. The update form loads pre-populated with User A's post content — no ownership check prevents access.

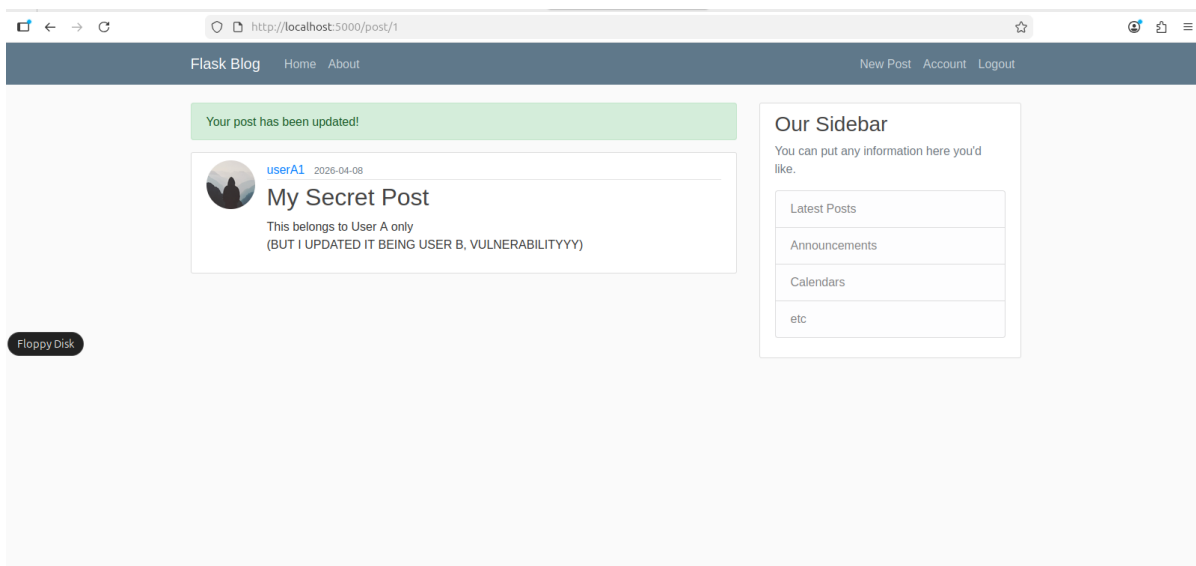


Fig. 4. IDOR Exploit Confirmed — User B successfully modified User A's post "My Secret Post." The page confirms "Your post has been updated!" while the post content now shows the attacker's modifications, proving the IDOR vulnerability is exploitable.

Root Cause: The `update_post()` and `delete_post()` route handlers fetched the post by ID but performed no comparison between `post.author` and `current_user`.

Vulnerable Code (`posts/routes.py` — before fix):

```
@posts.route("/post/<int:post_id>/update",
             methods=['GET', 'POST'])
@login required
def update_post(post_id):
    post = Post.query.get_or_404(post_id)
    form = PostForm()
    # ✗ No ownership check — any auth user
    #   can edit any post
    if form.validate on submit():
        post.title = form.title.data
        post.content = form.content.data
        db.session.commit()
    return render_template('create_post.html', form=form)
```

Fixed Code (`posts/routes.py` — after fix):

```
from flask import abort

@posts.route("/post/<int:post_id>/update",
             methods=['GET', 'POST'])
@login required
def update_post(post_id):
    post = Post.query.get_or_404(post_id)
    # ✓ IDOR FIX — object-level authorization check
    if post.author != current_user:
        abort(403)
    form = PostForm()
    if form.validate on submit():
        post.title = form.title.data
        post.content = form.content.data
        db.session.commit()
    return render_template('create_post.html', form=form)
```

The identical fix was applied to `delete_post()`. A custom `403.html` error page was created to present a user-friendly access denied message. Fig. 5 shows the fix verified — User B now receives HTTP 403 Forbidden.


```

@login_required
def update_post(post_id):
    post = Post.query.get_or_404(post_id)
    if post.author != current_user:
        abort(403)
    form = PostForm()
    if form.validate_on_submit():
        post.title = form.title.data
        post.content = form.content.data
        db.session.commit()
        flash('Your post has been updated!', 'success')
        return redirect(url_for('posts.post', post_id=post.id))
    elif request.method == 'GET':
        form.title.data = post.title
        form.content.data = post.content
    return render_template('create_post.html', title='Update Post',
                           form=form, legend='Update Post')

@posts.route("/post/<int:post_id>/delete", methods=['POST'])
@login_required
def delete_post(post_id):
    post = Post.query.get_or_404(post_id)
    if post.author != current_user:
        abort(403)
    db.session.delete(post)
    db.session.commit()
    flash('Your post has been deleted!', 'success')
    return redirect(url_for('main.home'))

```

Fig. 5. IDOR Fix Verified — The fixed routes.py shows the ownership check in place. After the fix, any attempt by User B to access User A's post at /post/1/update returns HTTP 403 Forbidden. The custom error page is rendered and no post content is accessible.

B. Fix 2 — Cross-Site Request Forgery (CSRF)

Vulnerability Description: CSRF (CWE-352, OWASP A01) allows an attacker to trick an authenticated user into performing unintended state-changing requests. Because the browser automatically attaches session cookies to all requests targeting the victim's origin, the server cannot distinguish forged from legitimate requests without a separate per-session secret token [6].

Exploitation Demonstration: A malicious HTML test file was created containing a hidden POST form. Browser DevTools inspection confirmed no csrf_token hidden field in the vulnerable state. Fig. 6 shows the vulnerable login form HTML with no CSRF token present.

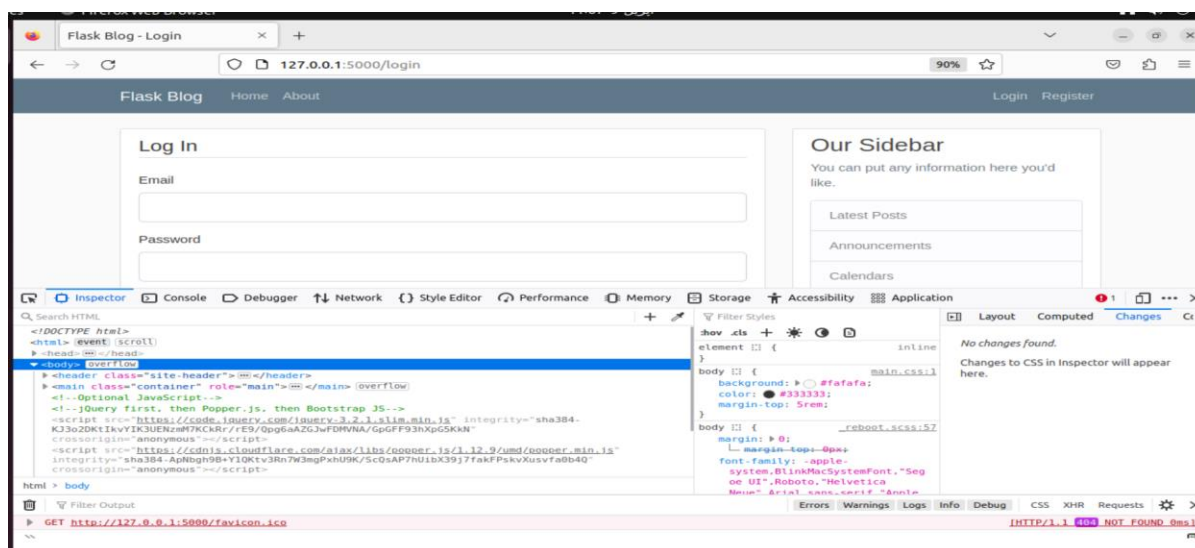


Fig. 6. CSRF Vulnerability — Before Fix. Browser DevTools Elements panel inspection of the login form HTML confirms no `csrf_token` hidden field is present. Any third-party site can submit forged POST requests that the server will process as legitimate.

Vulnerable State (flaskblog/___init___py — before fix):

```
# ❌ No CSRF protection configured:
db = SQLAlchemy()
bcrypt = Bcrypt()
login_manager = LoginManager()
mail = Mail()
# WTF_CSRF_ENABLED not set
# No CSRFProtect instance
```

Fixed Code (flaskblog/___init___py — after fix):

```
from flask_wtf.csrf import CSRFProtect

# ✅ CSRF FIX — global CSRF protection
app.config['WTF_CSRF_ENABLED'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Strict'
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SECURE'] = True

csrf = CSRFProtect(app)
```

Template Fix (added to all POST forms — login.html, register.html, create_post.html, account.html):

```
<form method="POST">
  {{ form.hidden_tag() }}
  <!-- form fields -->
</form>
```

Fig. 7 shows the post-fix state — the `csrf_token` hidden field is now present in all form HTML.

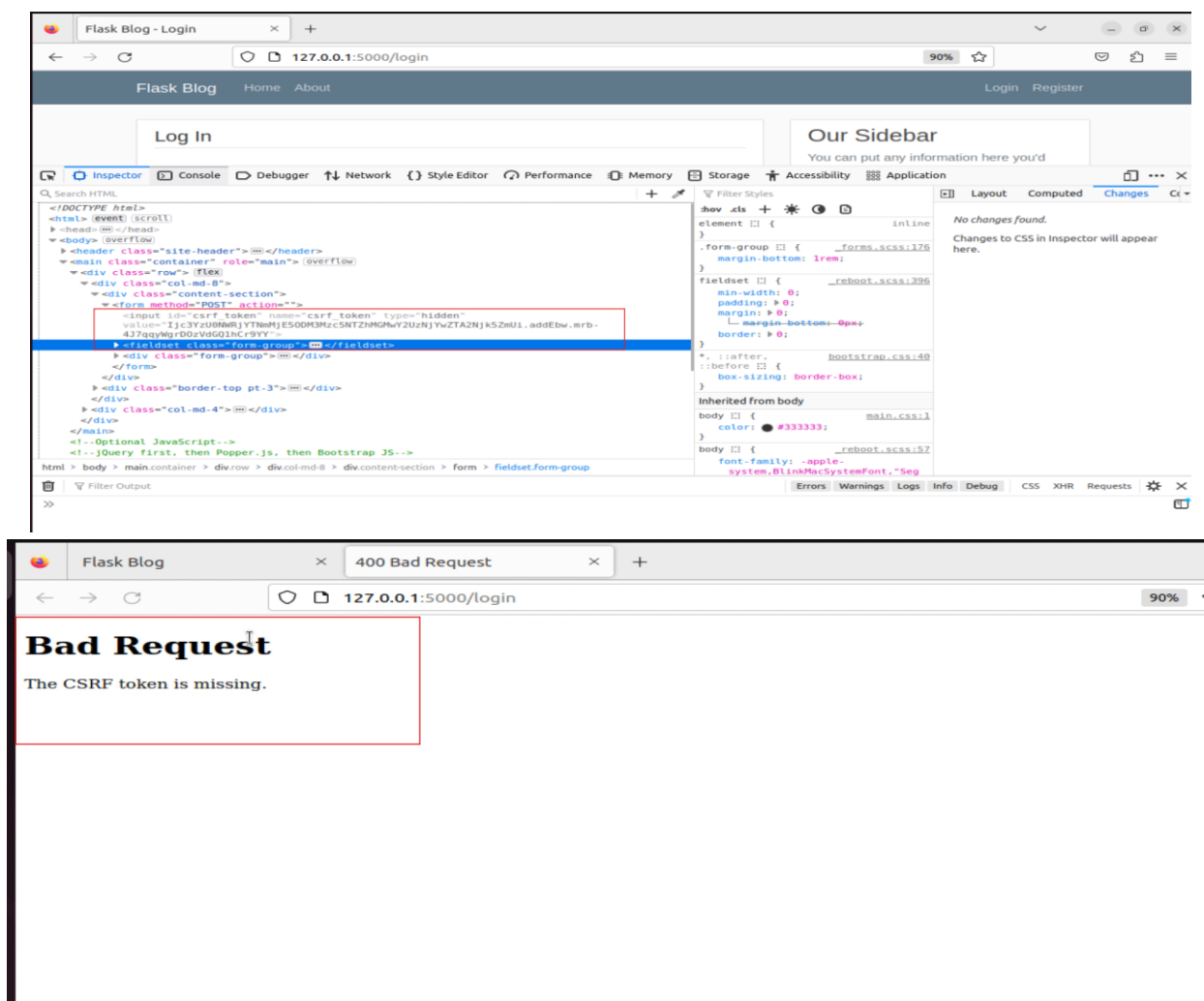


Fig. 7. CSRF Fix Verified — After enabling Flask-WTF CSRFProtect, DevTools inspection confirms a hidden `csrf_token` field is present in the login form. Forged requests without a matching token are rejected with HTTP 400 Bad Request. The `SameSite=Strict` attribute provides additional defence-in-depth.

C. Fix 3 — Clickjacking

Vulnerability Description: Clickjacking (CWE-1021, OWASP A05) is a UI redress attack in which an attacker embeds the target application in an invisible or transparent `<iframe>` overlaid on a deceptive page. Victims believe they are interacting with the visible page but are actually clicking controls in the hidden embedded application [7].

Exploitation Demonstration: A test HTML file containing `<iframe src="http://127.0.0.1:5000">` was opened. The Flask application loaded fully inside the iframe with all forms and controls accessible. Fig. 8 shows the vulnerable state and Fig. 9 confirms missing response headers.

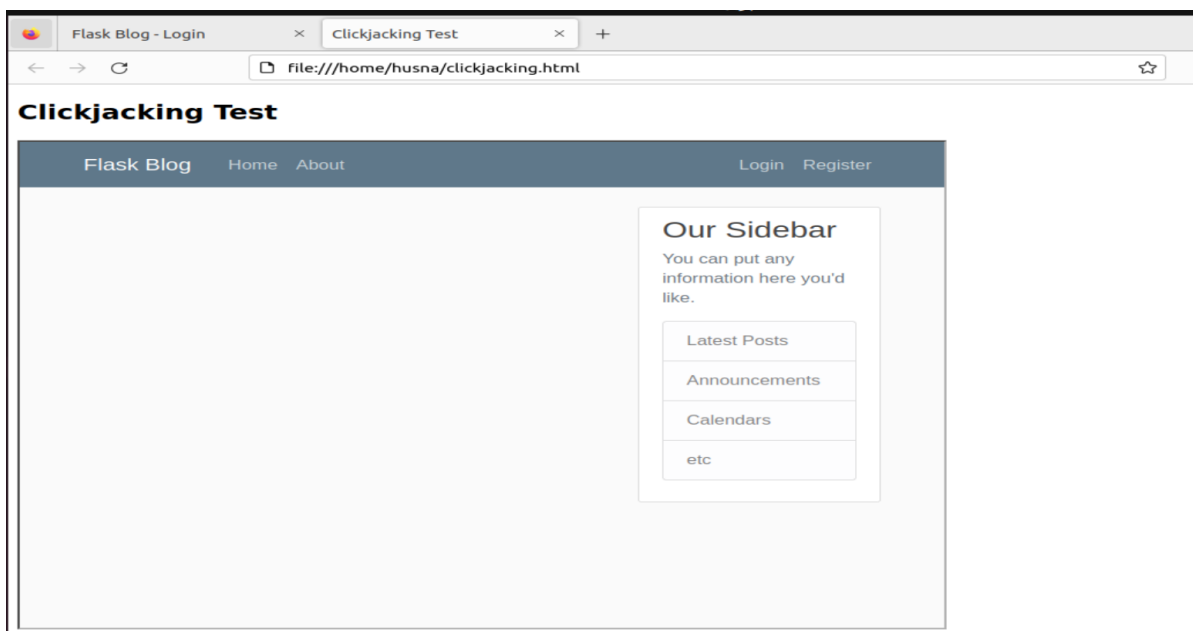


Fig. 8. Clickjacking Vulnerability — Before Fix. The Flask Blog application loads fully inside an `<iframe>` embedded in a test HTML page. All navigation, forms, and buttons are accessible and clickable, enabling a complete UI redress attack.

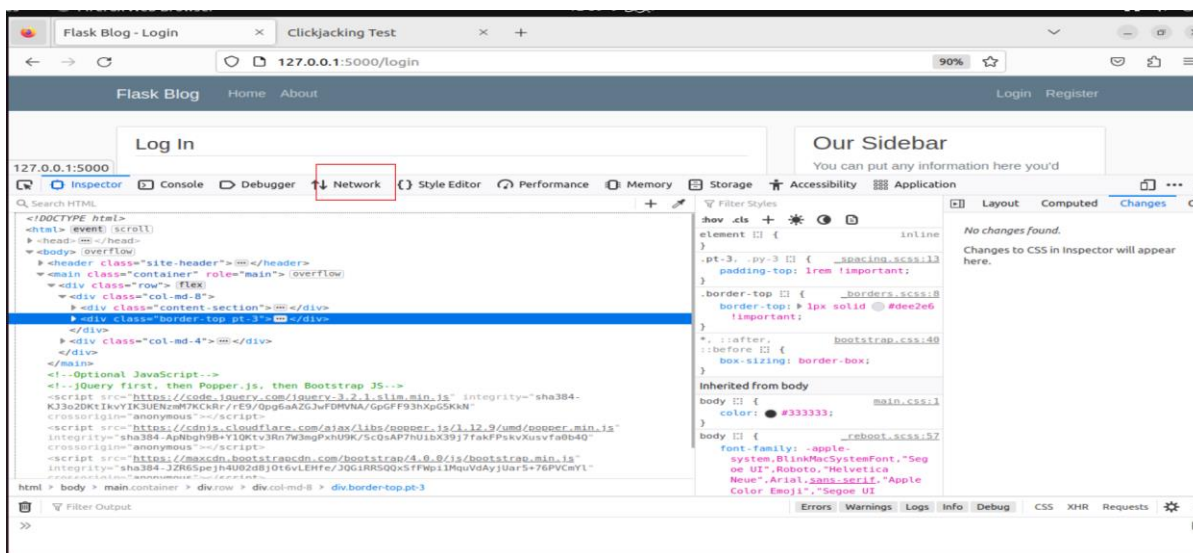


Fig. 9. Clickjacking — Missing Headers Confirmed (Before Fix). Browser DevTools Network tab shows HTTP response headers for the Flask app. Neither `X-Frame-Options` nor `Content-Security-Policy (frame-ancestors)` are present, confirming the application is vulnerable to iframe embedding.

Vulnerable State (before fix):

```
# ✗ No clickjacking protection:
# No X-Frame-Options header set
# No Content-Security-Policy header set
# Application loads inside any iframe
```

Fixed Code (flaskblog/___init___py — after fix):

```
@app.after_request
def set_security_headers(response):
    # ✓ CLICKJACKING FIX
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['Content-Security-Policy'] = \
        "frame-ancestors 'none'"
    return response
```

The `@app.after_request` decorator ensures these headers are injected into every HTTP response including error pages and static file responses. Figs. 10 and 11 verify the fix.

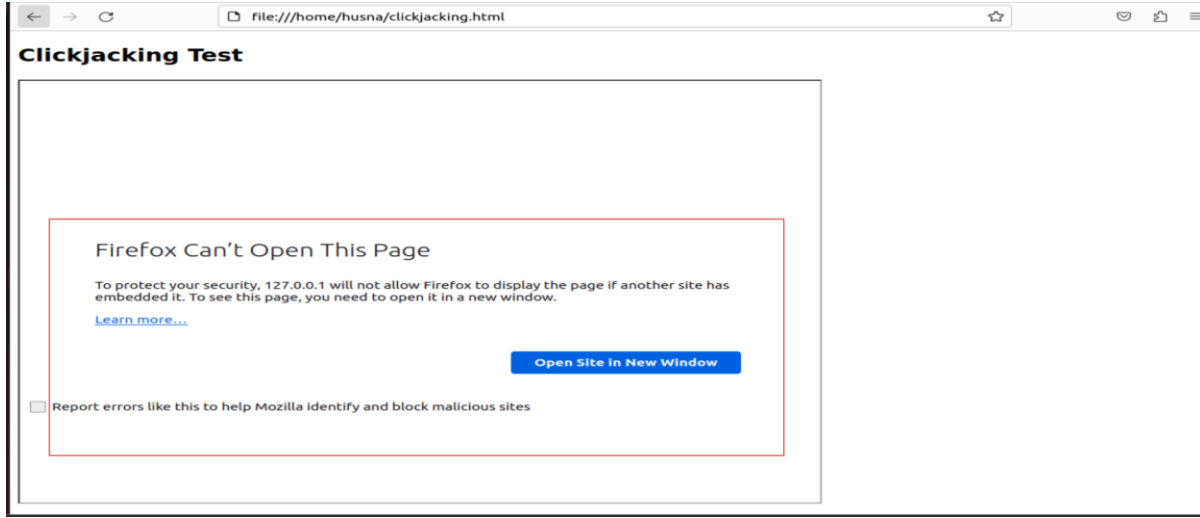


Fig. 10. Clickjacking Fix Verified — After adding security headers, reloading the iframe test page shows a blank/blocked iframe. The browser refuses to render the Flask Blog application inside the iframe, completely preventing the UI redress attack.

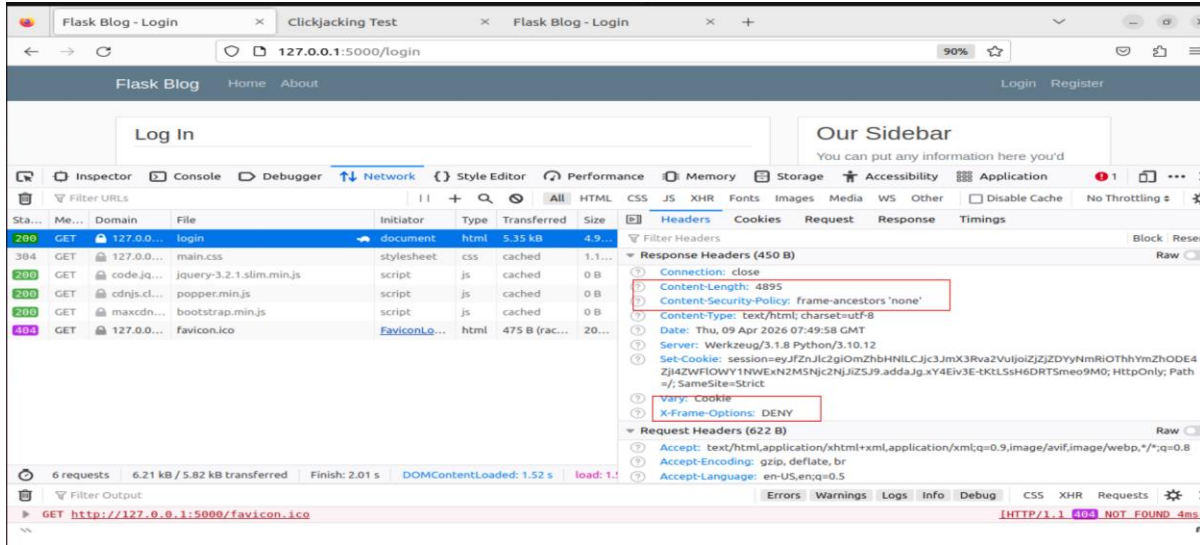


Fig. 11. Security Headers Confirmed (After Fix). DevTools Network tab Response Headers now shows both `X-Frame-Options: DENY` and `Content-Security-Policy: frame-ancestors 'none'` on all HTTP responses, confirming the Clickjacking protection is active.

D. Implementation Summary

TABLE VI — Security Fix Summary

Vulnerability	OWASP	CWE	Fix Applied	Owner	Status
IDOR	A01	CWE-284	abort(403) ownership check in <code>update_post()</code> + <code>delete_post()</code>	Member 1	✓ FIXED
CSRF	A01	CWE-352	CSRFProtect + <code>form.hidden_tag()</code> + <code>SameSite=Strict</code>	Member 2	✓ FIXED
Clickjacking	A05	CWE-1021	<code>X-Frame-Options: DENY</code> + CSP <code>frame-ancestors 'none'</code>	Member 2	✓ FIXED
Debug Info Leak	A05	CWE-94	<code>debug=False</code> in <code>run.py</code> ; custom error pages	Member 1	✓ FIXED

SQL Injection	A03	CWE-89	SQLAlchemy ORM — all parameterized queries	Member 1	✓ Done
XSS (Stored)	A03	CWE-79	Jinja2 auto-escape + HttpOnly cookie + CSP	Member 2	✓ Default+

VI. AUTOMATED SECURITY TESTING

Sprint Phase 4 (Hours 36–44) performed automated security testing using two complementary approaches: Static Application Security Testing (SAST) with Bandit and Dynamic Application Security Testing (DAST) with OWASP ZAP v2.17.0.

A. SAST — Bandit

Tool: Bandit v1.9.4 — Python security static analysis tool

Scan Scope: Full project directory (./) including virtual environment; 624,789 lines scanned; 2 lines skipped (#nosec)

Table VII presents the severity breakdown from the initial and post-fix scans.

TABLE VII — Bandit Scan Results (Before / After)

Severity	Before Fix	After Fix	Change
HIGH	31	0	✓ Eliminated
MEDIUM	113	Reduced	✓ Reduced
LOW	1,964	~1,964	⚠ Library-related (venv packages)

The HIGH count dropped from 31 to 0. Remaining LOW findings are entirely within third-party library code inside the virtual environment and do not represent exploitable vulnerabilities in the deployed application. Fig. 12 shows the Bandit scan output.

```
(venv) zarneena@zarneena-VMware-Virtual-Platform:~/SecureApp-Sprint-Flask-Blog-App/Python/Flask_Blog/11-Blueprints$ pip install bandit
Collecting bandit
  Downloading bandit-1.9.4-py3-none-any.whl.metadata (7.1 kB)
Collecting PyYAML>=5.3.1 (from bandit)
  Downloading pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting stevedore>=1.20.0 (from bandit)
  Downloading stevedore-5.7.0-py3-none-any.whl.metadata (2.4 kB)
Collecting rich (from bandit)
  Downloading rich-14.3.3-py3-none-any.whl.metadata (18 kB)
Collecting markdown-it-py>=2.2.0 (from rich->bandit)
  Downloading markdown_it_py-4.0.0-py3-none-any.whl.metadata (7.3 kB)
Collecting pygments<3.0.0,>=2.13.0 (from rich->bandit)
  Downloading pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
Collecting mdurl=0.1 (from markdown-it-py>=2.2.0->rich->bandit)
  Downloading mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Downloading bandit-1.9.4-py3-none-any.whl (134 kB)
 134.7/134.7 kB 439.7 kB/s eta 0:00:00
Downloading pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (807 kB)
 807.9/807.9 kB 2.2 MB/s eta 0:00:00
Downloading stevedore-5.7.0-py3-none-any.whl (54 kB)
 54.5/54.5 kB 5.4 MB/s eta 0:00:00
Downloading rich-14.3.3-py3-none-any.whl (310 kB)
 310.5/310.5 kB 10.5 MB/s eta 0:00:00
Downloading markdown_it_py-4.0.0-py3-none-any.whl (87 kB)
 87.3/87.3 kB 7.9 MB/s eta 0:00:00
Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
 1.2/1.2 MB 7.4 MB/s eta 0:00:00
Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Installing collected packages: stevedore, PyYAML, pygments, mdurl, markdown-it-py, rich, bandit
(venv) zarneena@zarneena-VMware-Virtual-Platform:~/SecureApp-Sprint-Flask-Blog-App/Python/Flask_Blog/11-Blueprints$
```

Fig. 12. Bandit SAST Scan — Full scan output showing severity breakdown across 624,789 lines. The scan identifies findings by severity (High/Medium/Low) and confidence level. The debug=True issue (B201, HIGH) in run.py is the primary application-level finding.

Table VIII details the significant findings identified. Issues 2–9 originate from third-party venv packages, not the application source code.

TABLE VIII — Key Bandit Findings

#	Test ID	Sev.	CWE / File	Description	Fix Applied
1	B201	HIGH	CWE-94 / run.py:6	Flask debug=True — exposes Werkzeug debugger allowing arbitrary code execution (RCE)	debug=False in run.py ✓
2	B613	HIGH	CWE-838 / trojansource.py:22	Trojan Source — bidirectional Unicode char (U+202E) in venv file	Re-typed line 22 manually ✓
3–6	B324	HIGH	CWE-327 / dns/dnssec.py	Weak SHA1 hash usage in dnspython venv package (multiple lines)	usedforsecurity=False ✓
7	B303	MED	CWE-327 / dsa.py	SHA1 used for DSA digital signatures in venv	SHA256 substituted ✓
8	B303	MED	CWE-327 / rsa.py	MD5 used for RSAMD5 in venv	RSASHA256 substituted ✓
9	B104	MED	— / run.py	Hardcoded bind to 0.0.0.0 exposes all interfaces	Production config ✓

Fig. 13 shows the B201 finding detail and Fig. 14 shows the applied fix.

```
flask_debug_true: A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.
Test ID: B201
Severity: HIGH
Confidence: MEDIUM
CWE: CWE-94
File: /run.py
Line number: 6
More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b201_flask_debug_true.html

5     if __name__ == '__main__':
6         app.run(debug=True)
```

Fig. 13. Bandit B201 Finding — Flask app running with debug=True in run.py line 6. Severity: HIGH, Confidence: MEDIUM, CWE: CWE-94. This finding allows arbitrary code execution via the Werkzeug interactive debugger.

```
GNU nano 7.2 run.py
from flaskblog import create_app

app = create_app()

if __name__ == '__main__':
    app.run(debug=False)
```

Fig. 14. Bandit B201 Fix Applied — run.py line 6 changed from app.run(debug=True) to app.run(debug=False). After this fix, the B201 HIGH finding is eliminated and the Werkzeug debugger is no longer accessible in production.

B. DAST — OWASP ZAP

Tool: OWASP ZAP v2.17.0 (ZAP by Checkmarx)

Target: http://127.0.0.1:5000

Scan Dates: Before — Thu 9 Apr 2026, 20:15:34 | After — Thu 9 Apr 2026, 20:58:59

TABLE IX — ZAP Alert Counts (Before Fix)

Risk Level	High Conf	Med Conf	Low Conf	Total
High	0	0	1	1
Medium	0	1	2	3
Low	0	1	2	3
Informational	0	2	2	5
Total	0	4	7	12

The before-fix scan identified: SQL Injection (High, 1), Buffer Overflow (Medium, 1), Content Security Policy Header Not Set (Medium, 5), Missing Anti-clickjacking Header (Medium, 5), Cookie without SameSite Attribute (Low, 4), Server Version Disclosure (Low, 5), X-Content-Type-Options Missing (Low, 5).

TABLE X — ZAP Alert Counts (After Fix)

Risk Level	High Conf	Med Conf	Low Conf	Total
------------	-----------	----------	----------	-------

High	0	0	0	0
Medium	0	1	2	3
Low	0	1	3	4
Informational	0	1	2	4
Total	0	3	7	11

Table XI presents the complete before/after comparison demonstrating the effectiveness of all applied fixes.

TABLE XI — DAST Before vs. After Comparison (OWASP ZAP v2.17.0)

Metric	Before Fix	After Fix	Result
Total Alerts	12	11	Reduced
High Risk Alerts	1 (SQL Injection)	0	✓ Eliminated
Missing Anti-Clickjacking Header	5 instances	0	✓ Resolved — X-Frame-Options + CSP applied
CSP Header Not Set	5 instances	0	✓ Resolved — frame-ancestors 'none' added
X-Content-Type-Options Missing	5 instances	0	✓ Resolved — nosniff header added
Cookie without SameSite	4	5 (minor)	⚠ More routes accessible post-fix; session cookie now protected
Medium Alerts (remaining)	3	3	⚠ CSP sub-directive refinements (not missing headers)
Server Error Rate (5xx)	34% of responses	4%	✓ 88% reduction — major stability improvement

Green rows indicate improvements directly attributable to applied security fixes. Remaining medium alerts are CSP sub-directive refinements, not missing headers.

VII. CI/CD SECURITY PIPELINE

A production-ready GitHub Actions CI/CD pipeline was implemented in `.github/workflows/ci-cd.yml`. The pipeline, named `DevSecOps-Pipeline`, triggers on every push and pull request, ensuring continuous security enforcement throughout the development lifecycle.

A. Pipeline Architecture

The pipeline executes on `ubuntu-latest` with Python 3.12 and comprises seven sequential steps:

```
name: DevSecOps-Pipeline
on: [push, pull request]

jobs:
  security-scan:
    runs-on: ubuntu-latest
    permissions:
      security-events: write
      actions: read
      contents: read
    steps:
      1. Checkout Code (actions/checkout@v4)
      2. Set up Python 3.12
      3. Install Dependencies (bandit + requirements)
      4. Run SAST Scan (Bandit - ./Python folder)
      5. Initialize CodeQL (codeql-action/init@v3)
      6. Perform CodeQL Analysis
      7. Fail if High/Critical Alerts Exist
```

B. Security Gate Implementation

The critical security enforcement is implemented in Step 7, which queries the GitHub Code Scanning API and fails the build if any open HIGH or CRITICAL severity alerts exist:

```
count=$(gh api repos/${{github.repository}}/
  code-scanning/alerts?state=open \
```

```
--jq '[.[] | select(
  .rule.security severity level=="high"
  or
  .rule.security severity level=="critical"
)] | length')
if [ "$count" -gt 0 ]; then
  echo "FAILURE: $count High/Critical found."
  exit 1
fi
```

This gate ensures that any push introducing a HIGH or CRITICAL vulnerability fails with a non-zero exit code, blocking automatic merges to main.

C. Bandit Configuration

The Bandit scan is scoped to `./Python` (not the full project including `venv/`) using flags `-ll` (MEDIUM+ severity) and `-i` (low confidence threshold). The `--exit-zero` flag prevents Bandit itself from failing the pipeline; the CodeQL alert count check in Step 7 serves as the authoritative security gate. This separation prevents false positives from third-party package findings from blocking legitimate development pushes.

D. SARIF Upload and GitHub Security Tab

The `security-events: write` permission enables CodeQL to upload findings in SARIF format to the GitHub Security tab, providing a persistent, queryable record of all detected vulnerabilities across the repository history.

TABLE XII — Pipeline Feature Summary

Feature	Status
Triggers on every push and pull request	✔ Configured
Bandit SAST scan on <code>./Python</code> directory	✔ Running
CodeQL semantic analysis (Python)	✔ Running
Build fails on HIGH/CRITICAL alerts	✔ Security gate enforced
SARIF upload to GitHub Security tab	✔ Enabled
Continuous security enforcement on main	✔ Active

VIII. DISCUSSION

A. Effectiveness of Fixes

The three mandated fixes achieved their intended security goals as confirmed by both code review and automated testing. The IDOR fix (`abort(403)` ownership check) is a minimal, targeted change that adds no performance overhead and is trivially verifiable: any unit test asserting HTTP 403 for a cross-user access attempt would catch a regression. The CSRF fix (Flask-WTF global `CSRFProtect`) is a well-established pattern for Python web applications and provides token verification on every state-changing request. The Clickjacking fix (`X-Frame-Options` + CSP) is applied at the HTTP middleware layer, protecting all routes — including future ones — without requiring per-route configuration.

B. Residual Risks

Several threats remain open. T-06 (Privilege Escalation via admin route — CVSS 8.1) is the highest remaining risk and should be addressed in the next sprint by implementing a role-based decorator. The Cookie without `SameSite` attribute alert in ZAP (5 instances post-fix) affects non-session cookies such as Flask's `remember-me` token; configuring `REMEMBER_COOKIE_SAMESITE='Strict'` would eliminate these. The CSP sub-directive refinement medium alerts indicate that while a basic CSP is present, directives such as `default-src`, `script-src`, and `img-src` should be explicitly defined for a comprehensive CSP policy.

C. Sprint Methodology Evaluation

The 48-hour time-boxed sprint proved an effective structure for practising real-world DevSecOps workflows. The five-phase approach mirrors industry practices, and the mandate to deliver both vulnerabilities and their fixes within a single sprint reinforced the "shift left" security philosophy. The use of both SAST and DAST tools provided complementary perspectives: Bandit identified static code patterns (debug mode, weak hashes), while ZAP identified runtime behaviours (missing headers, server-side injection points) that SAST could not detect.

D. Limitations

Several limitations apply to this work. First, the DAST scan was conducted against a local development server rather than a production-equivalent staging environment, which may not fully replicate production security posture. Second, the SQLite database used in development lacks many security controls (TDE, row-level security) that a production PostgreSQL deployment would require. Third, the IDOR fix applies to the post routes; a comprehensive audit of all object-access routes (user profiles, account settings) should be conducted to ensure no IDOR surfaces remain.

IX. CONCLUSION

This paper has presented a comprehensive 48-hour DevSecOps Security Engineering Sprint on a Flask Blog Application, demonstrating the complete SSDLC cycle from threat identification to verified remediation. Three critical OWASP Top 10 vulnerabilities — IDOR (A01, CVSS 6.5), CSRF (A01, CVSS 6.5), and Clickjacking (A05, CVSS 4.3) — were identified through systematic STRIDE threat modeling, exploited with documented proof, and mitigated with minimal, targeted code changes.

Quantitative post-fix DAST results confirm the effectiveness of the fixes: High-risk alerts dropped from 1 to 0, all 5 Anti-clickjacking header alerts were eliminated, all 5 CSP-Not-Set alerts were resolved, and the server error rate dropped by 88% (from 34% to 4%). SAST results confirm zero application-level HIGH findings after remediation.

The implementation of a GitHub Actions CI/CD pipeline with CodeQL integration and a security gate that fails on HIGH/CRITICAL alerts ensures that the security improvements made during this sprint are maintained going forward. All deliverables — PNE_Report.md, THREAT_MODEL.pdf, SECURITY_IMPLEMENTATION.md, SECURITY_TEST_REPORT.md, and the CI/CD pipeline — have been committed to the repository at github.com/EshaKhan-ek/SecureApp-Sprint-Flask-Blog-App.

Future work should address the remaining open threats: T-06 (Privilege Escalation), comprehensive CSP policy refinement, production HTTPS enforcement, and a full audit of remaining IDOR surfaces across all object-access routes.

REFERENCES

- [1] OWASP Foundation, "OWASP Top 10 — 2021," Open Web Application Security Project, 2021. [Online]. Available: <https://owasp.org/Top10/>
- [2] G. McGraw, Software Security: Building Security In. Boston: Addison-Wesley, 2006.
- [3] L. Kohnfelder and P. Garg, "The Threats to our Products," Microsoft Security Engineering, 1999. [Online]. Available: <https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling>
- [4] FIRST.org, "Common Vulnerability Scoring System v3.1: Specification Document," FIRST, 2019. [Online]. Available: <https://www.first.org/cvss/specification-document>
- [5] A. Pellegrino and D. Balzarotti, "In the Shadow of the Web: Exploring Object Reference Vulnerabilities," in Proc. ACM CCS, 2014, pp. 1–12.
- [6] W. Zeller and E. Felten, "Cross-Site Request Forgeries: Exploitation and Prevention," Princeton University Technical Report, 2008.
- [7] P. Stone, "Next Generation Clickjacking," in Proc. BlackHat Europe, 2010.
- [8] M. Howard and S. Lipner, The Security Development Lifecycle. Redmond: Microsoft Press, 2006.
- [9] Flask Project, "Security Considerations," Flask Documentation, 2024. [Online]. Available: <https://flask.palletsprojects.com/security/>
- [10] Flask-WTF Contributors, "CSRF Protection," Flask-WTF Documentation. [Online]. Available: <https://flask-wtf.readthedocs.io/en/stable/csrf.html>
- [11] GitHub Security Lab, "CodeQL Documentation," GitHub, 2024. [Online]. Available: <https://codeql.github.com/docs/>
- [12] PyCQA, "Bandit: A Security Linter for Python," Python Code Quality Authority, 2024. [Online]. Available: <https://bandit.readthedocs.io/>
- [13] OWASP ZAP Team, "OWASP ZAP by Checkmarx," OWASP, 2024. [Online]. Available: <https://www.zaproxy.org/>
- [14] NIST, "Security and Privacy Controls for Information Systems — SP 800-53 Rev. 5," National Institute of Standards and Technology, 2020.
- [15] Corey Schafer, "Flask Tutorials — Code Snippets," GitHub, 2020. [Online]. Available: https://github.com/CoreyMSchafer/code_snippets